

Electric Vehicle Charging Simulator User Manual

Version	
Product	v0.22.13
Manual	Rev1

- Electric Vehicle Charging Simulator User Manual
 - Introduction
 - Installation
 - First Run Default Configurations
 - First Run Define Configurations
 - First Run
 - Use the Application
 - Change Configurations
 - By Flags
 - By Restful API
 - /configs/general
 - /configs/simulators
 - Setup Simulators
 - Manage Models
 - /simulators/models
 - /simulators/models/:id
 - Manage Simulator
 - /simulators
 - /simulators/:id
 - Run The Simulators
 - /simulators/run
 - /simulators/stop
 - /simulators/status
 - Health Check
 - Appendix
 - Flags
 - Log Level [logLevel]
 - Asset Type [type]
 - Protocols [protocol]
 - Phase Rotation [phaseRotation]
 - Current Type [currentType]
 - Charging States [chargingState]
 - Error Code [errorCode]
 - Modbus Keys

Introduction

The Application Manual contains all the essential information for the user to make full use of the system. This document contains the description of the application functions and capabilities, installation, modes of operation, and how to use it. The manual format and steps may be altered during the development of the application.

Installation

There are no installation file the application will do the necessary actions in the first execution to setup. There are a some files and folders that need to be in place to execute the application, for them to be created the user that executes the application need to have write permissions to the folder where the application file is located, and to the folder(s) where we want to locate the other files.

The app doesn't have any GUI but in the other hand has a **Restful API** that enables the user to control the application, and have some arguments that can be set during the execution of the app.

First Run Default Configurations

The first time the application is executed, and without any arguments, it will generate the `configs.json` file, the `settings` folder and in it the `general.json` file, the `simConf` folder, and the `/simConf/models` folder. The previous files and folders are essential for the application to run and can't be moved or deleted. Some of this configurations are set using the default values defined in the application and can be changed on the first execution or during the usage of the application.

First Run Define Configurations

To know the configurations that can be changes on the execution we can run the application with the flag `-h`, `./ev-chargers-simulator-linux-amd64 -h`, that will print the list of all the flags the are supported as all the default values.

If you whant to change any of the listed configurations ([Appendix](#)) during the 1st execution, you can simply set the *flag* and the value after.

Change the **General configurations folder**:

- `./ev-chargers-simulator-linux-amd64 -gs /full/path/to/the/location`

Change the **Rest API http server port**:

- `./ev-chargers-simulator-linux-amd64 -ap 3333`

Change the **General and Simulators folder**:

- `./ev-chargers-simulator-linux-amd64 -gs /full/path/to/the/location -ss /full/path`

If only part of the *flags* are set in the 1st run the application will set the other configurations with the default values, as in the previous section, that are shown when the user execute the application using the *flag -h*.

First Run

After the user start the first time the application it will have the same behaviour as the rest of the executions. It will start the *Restful API Server*, load the configurations, look for the the *models* and *simulators* configuration files and load them, and will wait for the user commands using the *API*.

Use the Application

There are a couple of information's that we need to know to be able to use this application to it's full potential, as how to change the configurations, create/edit/remove models and simulators, what is a model, etc... In this section we will dive into explaining how to use it, and the interfaces that are available to the user.

Change Configurations

There are multiple configurations some of them are not changeable but others are, and the user has two ways to change them.

By Flags

As described in the [Installation](#) there are some *command line* arguments that can be used to define the configurations in the first execution, the full list of arguments can be found in the [Appendix Flags](#) section.

After the *installation* process we can still use the *flags* to change the configurations, but to do it we need to use an extra one `-i`. This is the argument that triggers the application to save the new information.

With this we can run the application and at the same time change the configurations and save them for the next execution. The next examples are the same the were written previously but with the `-i` flag, so the configurations are changed and saved.

Change the **General configurations folder**:

- `./ev-chargers-simulator-linux-amd64 -i -gs /full/path/to/the/location`

Change the **Rest API http server port**:

- `./ev-chargers-simulator-linux-amd64 -i -ap 3333`

Change the **General and Simulators folder**:

- `./ev-chargers-simulator-linux-amd64 -i -gs /full/path/to/the/location -ss /full/path`

By Restful API

Executing the application it will start an *HTTP* service that will enable changing the configurations. There are two endpoints that can be used to change the application configurations, the `/configs/general` and `/configs/simulators`.

`/configs/general`

With this endpoint we can get the current configuration as change them. The list of all the configurations that can be changed are listed down.

- `hostIp`: The IP address of the computer that is running the application
- `connTimeout`: Connection timeout for all the simulators
- `cSAddr`: OCPP central system IP address

- **cSPort**: OCPP central system port that is listening for the charge points
- **lang**: Language
- **apiAddr**: IP address that the *HTTP* service will run on
- **apiPort**: IP port that the *HTTP* service will be listening on

To list the current configurations values execute the request:

```
GET http://{apiAddr}:{apiPort}/configs/general
```

The response will return all the configurations listed previously in a *JSON* format.

The same format returned by the list request should be used to execute the update configurations request.

```
PUT http://{apiAddr}:{apiPort}/configs/general
{
  "hostIp": "0.0.0.0",
  "connTimeout": 70,
  "cSAddr": "localhost",
  "cSPort": "1234",
  "lang": "pt-PT",
  "apiAddr": "localhost",
  "apiPort": "8001"
}
```

WARNING: The update of any of the configurations and the update of the other endpoints needs to have the full body, could not be a partial update. If only one field needs to be changed the full body needs to be sent.

/configs/simulators

In this endpoint we can change the path where the configuration files of the simulators will be located.

To get the current path we can use the:

```
GET http://{apiAddr}:{apiPort}/configs/simulators
```

To update as in the previous endpoint we can use the *JSON* response structure.

```
PUT http://{apiAddr}:{apiPort}/configs/simulators
{
  "simFolder": "/full/path/to/folder/"
}
```

Setup Simulators

In this section it is described how to setup and use the simulators. In opposition to the previous section the only way to do this is interfacing with the *Restful API*.

Manage Models

The **Models** are used to add configurations to the simulators, for example the *OCPP Chargers* have information that are retrieved by the central system that is related with the equipment. For the *OCPP Chargers* in the boot request the charger sends the brand, model, etc... and this type of information is configured in the **Model**. The properties are listed bellow:

- **id**: Model identifier
- **name**: Model name
- **type**: Type of asset that this config can be used (ocpp, modbus)
- **ca**: CA certificate location and name, full path
- **cert**: Client certificate location and name, full path
- **key**: Client certificate key location and name, full path
- **basicAuth**: HTTP basic authentication credentials
 - **username**: HTTP basic authentication username
 - **password**: HTTP basic authentication password
- **ocpp**: Ocpp structure
 - **serialNumb**: Equipment serial number
 - **model**: Equipment model name
 - **vendor**: Equipment vendor
 - **vendorId**: Vendor identifier
 - **fwVersion**: Firmware version
 - **meterSerialNumb**: Power meter serial number
 - **modem**: Modem information
 - **iccid**: SIM card identifier
 - **imsi**: International Mobile Subscriber Identity
- **modbus**: Modbus structure
 - **coils**: Coils mapping
 - **discrete**: Descrete inputs mapping
 - **holdingRegisters**: Holding registers mapping
 - **inputRegisters**: Input registers mapping
 - **addresses**: List of modbus addresses mapping

To use a *model* it needs to be configured in the *simulator* configuration. One simulator can be associated with one *model* but one *model* can have multiple *simulators*.

When creating a model for the **modbus** type the mapping defined in the **addresses** property, should be a string with the address and a string with the value to be returned. There are some *keys* that can be used to set in the mapping that instead of returning a static value will call a function that will generate the response value. The *keys* list can be found in the [Modbus Keys appendix](#).

/simulators/models

This endpoint gives the ability to list and create *models*.

To list the existing models use the following method:

```
GET http://{apiAddr}:{apiPort}/simulators/models
```

It will return the total of existing *models* and the list of them.

```
{
  "total": 1,
  "models": [
    {
      "id": "3dfb64f3-d26b-44f5-8b1a-c56b56fa1b15",
      "name": "model1",
      "type": "ocpp",
      "ocpp": {
        "serialNumb": "",
        "model": "",
        "vendor": "",
        "vendorId": "",
        "fwVersion": "",
        "meterSerialNumb": "",
        "modem": {
          "iccid": "",
          "imsi": ""
        }
      },
      "modbus": {}
    }
  ]
}
```

Create a new *model* with the same endpoint but now using the **POST** method:

```
POST http://{apiAddr}:{apiPort}/simulators/models
{
  "name": "model1",
  "type": "ocpp",
  "ocpp": {
    "serialNumb": "qweqwe",
    "model": "EV Charger 1",
    "vendor": "Simulator",
    "fwVersion": "1.0.0",
    "meterSerialNumb": "789asd",
    "modem": {
      "iccid": "111111111111",
      "imsi": "222222222222"
    }
  }
}
```


The service will return the same body structure sent in the request, but with the **ID** generated.

The properties **name** and **type** are required, the **name** is a free field but the **type** is not. It can only have the values **ocpp** or **modbus**, and it will be used to handle the type of protocol the the associated simulators can have.

/simulators/models/:id

Besides listing and creating the *models*, the user has the ability to edit and delete a model. This endpoint has the same path as the previous one but with the **ID (:id)** at the end that will identify the *model* that will be edited or deleted.

To edit a model get the **ID**, add it to the end of the endpoint path, and the full body of the *model* with the edited values.

```
PUT http://{apiAddr}:{apiPort}/simulators/models/{model-identifier-here}
{
  "name": "model1",
  "type": "modbus",
  "modbus": {
    "addresses": {
      "512": "0",
      "513": "6",
      "514": "powerImportKw"
    }
  }
}
```

Now deleting a *model* use the following:

```
DELETE http://{apiAddr}:{apiPort}/simulators/models/{model-identifier-here}
```

Manage Simulator

Now looking into the core of this application, the **Simulators**. There is on endpoint that is used to manage the **Simulator** assets and their configurations. In here will explain how to add, edit, delete, and configure the assets. All the configurable properties are listed bellow:

- **name**: Simulator name
- **LogLevel**: Log level of the simulator, the list of accepted values are info, error, warn, panic, fatal, debug
- **type**: Type of the asset that the configuration will be used to (evc, pm)
- **protocol**: Protocol used by the asset
- **tls**: Set if the asset will connect using TLS or not, the default is false (if true the model is required)

- **basicAuth**: Set if the asset will use the basic http auth to connect, the default is false (if true model is required)
- **model**: Id of the configuration file set in the model's folder
- **port**: Communication ip port
- **cPid**: Charge point id to identify the unit (used in the ocpp protocol)
- **startCharging**: Set the asset to start charging behaviour by itself
- **phases**: Phases number
- **phaseRotation**: The asset phase rotation, if the asset is DC the value should be NotApplicable
- **currentType**: Type of current of the asset (ac or dc)
- **authorizeRemote**: Configuration AuthorizeRemoteTxRequests
- **authList**: Enable or disable authorization local list
- **evses**: List of evses that the asset has
 - **id**: Evse identifier number
 - **connectors**: The list of connectors of the evse
 - **id**: Connector identifier number
 - **data**: The loop of data
 - **duration**: The duration in seconds that the current data will be in place
 - **chargingState**: The state of charging for the current data
 - **errorCode**: Error code
 - **powerFactor**: The power factor
 - **power**: Power in W
 - **powerExport**: Power exported by the ev to the cp in W, vehicle to grid
 - **voltage**: Array of voltages each entry for each phase in V
 - **startSoC**: Start ev charge state in percentage
 - **endSoC**: End ev charge state in percentage

The *OCPP v1.6* protocol doesn't support multiple EVSEs in the same charge point. This rule was set in place for the creation and update of the assets with the type **evc** that have the **ocpp16** protocol, when both of this properties match the asset can only have **one** item in the **evses** list. Trying to create or update an asset with more than **one** EVSE, with the type and protocol matching what was described before, the api will return an error message.

/simulators

This endpoint can be used to create and list all the *simulators*.

To list the existing *simulators* use the following method:

```
GET http://{apiAddr}:{apiPort}/simulators
```

It will return the total number of number of existing *simulators* and their list.

```
{
  "total": 1,
  "assets": [
    {
```

```

"simId": "585e8e82-b296-49f7-aea7-f3478a0c3a51",
"name": "sim1",
"type": "evc",
"protocol": "ocpp16",
"model": "000000000-0000-0000-0000-000000000000",
"cPid": "123789",
"startCharging": false,
"phases": 1,
"phaseRotation": "NotApplicable",
"currentType": "dc",
"authorizeRemote": true,
"authList": true,
"evses": [
  {
    "id": 1,
    "connectors": [
      {
        "id": 1,
        "omitEmpty": false,
        "data": [
          {
            "duration": 10,
            "chargingState": 1,
            "errorCode": 0,
            "powerFactor": 900,
            "power": 0,
            "powerExport": 0,
            "voltage": [
              230,
              0,
              0
            ],
            "startSoC": 0,
            "endSoC": 0
          }
        ]
      }
    ]
  }
]
}

```

To create a new *simulator* use the same endpoint but with a new method and sending the body described down. There are a couple of rules that are required to be followed so the application can work normally, the **data** array field in the configuration needs to have the 1st position with the **Available** (1) state and the last position with **Finishing** (6). The **chargingState** available values are exposed in the [Charging States appendix](#).

```

POST http://{apiAddr}:{apiPort}/simulators
{
  "name": "sim1",
  "type": "evc",
  "protocol": "ocpp16",
  "model": "000000000-0000-0000-0000-000000000000",
  "cPid": "123789",
  "startCharging": false,
  "phases": 1,
  "phaseRotation": "NotApplicable",
  "currentType": "dc",
  "authorizeRemote": true,
  "authList": true,
  "evses": [
    {
      "id": 1,
      "connectors": [
        {
          "id": 1,
          "omitempty": false,
          "data": [
            {
              "duration": 10,
              "chargingState": 1,
              "errorCode": 0,
              "powerFactor": 900,
              "power": 0,
              "powerExport": 0,
              "voltage": [
                230,
                0,
                0
              ],
              "startSoC": 0,
              "endSoC": 0
            }
          ]
        }
      ]
    }
  ]
}

```

It will return the created simulator information as response.

/simulators/:id

With this endpoint we are able to edit the configuration of one simulator as able to delete one. To use it replace the `:id` by the identifier of the *simulator* that is present in the property `simId`.

To update a simulator use the following example, to update any of the properties the full *simulator* body.

```

PUT http://{apiAddr}:{apiPort}/simulators/{simulator-identifier-here}
{
  "name": "sim1",
  "type": "evc",
  "protocol": "ocpp16",
  "model": "000000000-0000-0000-0000-000000000000",
  "cPid": "123789",
  "startCharging": false,
  "phases": 1,
  "phaseRotation": "NotApplicable",
  "currentType": "dc",
  "authorizeRemote": true,
  "authList": true,
  "evses": [
    {
      "id": 1,
      "connectors": [
        {
          "id": 1,
          "omitempty": false,
          "data": [
            {
              "duration": 10,
              "chargingState": 1,
              "errorCode": 0,
              "powerFactor": 900,
              "power": 0,
              "powerExport": 0,
              "voltage": [
                230,
                0,
                0
              ],
              "startSoC": 0,
              "endSoC": 0
            }
          ]
        }
      ]
    }
  ]
}

```

To delete a simulator use the *delete* method:

```
DELETE http://{apiAddr}:{apiPort}/simulators/{simulator-identifier-here}
```

Run The Simulators

After setting all up following the previous sections of the manual the application can now be used to perform intended action. There are some endpoints that are available and described down that will start, stop, and show the status of the emulators.

/simulators/run

To start all the *emulators* assets execute a request to the endpoint described bellow, this endpoint will start all the emulators created following the configurations defined.

```
POST http://{apiAddr}:{apiPort}/simulators/run
```

/simulators/stop

To stop the *emulators* execute the following request, that will stop all the running assets.

```
POST http://{apiAddr}:{apiPort}/simulators/stop
```

/simulators/status

During the application execution there is a way to retrieve some information about the status of all of the running EV Chargers. With the following endpoint the assets are returned in the response body with the state of each.

- **total**: Total number of assets
- **assets**: List of assets
 - **id**: Asset identifier
 - **name**: Asset name
 - **state**: State of the asset (active, inactive)
 - **power**: Current asset power
 - **energy**: Energy consumption of the execution

```
GET http://{apiAddr}:{apiPort}/simulators/status
```

Bellow is an example of the response body that will be returned:

```
{
  "total": 2,
  "assets": [
    {
      "id": "585e8e82-b296-49f7-aea7-f3478a0c3a51",
      "name": "sim1",
      "state": "active",
      "power": 0,
```

```
        "energy": 0
      },
      {
        "id": "585e8e82-b296-49f7-aea7-f3478a0c3a52",
        "name": "sim2",
        "state": "active",
        "power": 99999.9999996362,
        "energy": 2307.657838991605
      }
    ]
  }
}
```

Health Check

Since the application can be run as a service, where the *simulators* can be started and stop as needed this endpoint was created to enable the ability to know if the service is running or not. In case the need to know if it is running, the application not the *emulators*, call the following endpoint.

```
GET http://{apiAddr}:{apiPort}/health
```

If the application is running will return the following body, in case it is not it will fail since the endpoint is not reachable.

```
{
  "message": "OK"
}
```

Appendix

Flags

```
-ai string
    Address used to serve the api http server (default "localhost")
-ap string
    Port used to serve the api http server (default "8000")
-csi string
    Central system ip address (default "iot-gate-imx8.lan")
-csp string
    Central system port (default "49443")
-gs string
    General configuration folder, store the application general
configurations (default "/home/alex/Downloads/ev-chargers-simulator-linux-
amd64/build/settings")
-i      Force the update of the configurations with the values in the flags
-l string
    Language used by the application [en-GB, pt-PT] (default "en-GB")
-s string
    Simulator host ip address (default "84.5.7.64")
-ss string
    Simulators configuration folder, store the simulators
configurations (default "/home/alex/Downloads/ev-chargers-simulator-linux-
amd64/build/simConf")
-t int
    Connection timeout (seconds) (default 70)
-v      Return the current application version
```

Log Level [logLevel]

- **info**: Only logs information messages
- **error**: Only logs error messages
- **warn**: Only logs warnings messages
- **panic**: Only logs panic messages
- **fatal**: Only logs fatal messages
- **debug**: Logs every message type

Asset Type [type]

- **evc**: Type electric vehicle charger
- **pm**: Type power meter (only supported for modbus)

Protocols [protocol]

- **ocpp16**: Ocpp version 1.6
- **ocpp201**: Ocpp version 2.0.1 (limited)
- **modbus**: Modbus protocol (only supported for pm)

Phase Rotation [phaseRotation]

- **NotApplicable**: Not applicable for dc chargers
- **Unknown**: Not able to retrieve the rotation
- **RST**: L1 L2 L3
- **RTS**: L1 L3 L2
- **SRT**: L2 L1 L3
- **STR**: L2 L3 L1
- **TRS**: L3 L1 L2
- **TSR**: L3 L2 L1

Current Type [currentType]

- **ac**: Alternating current
- **dc**: Direct current

Charging States [chargingState]

- **1**: Available
- **2**: Preparing
- **3**: Charging
- **4**: SuspendedEV
- **5**: SuspendedEVSE
- **6**: Finishing
- **7**: Reserved
- **8**: Unavailable
- **9**: Faulted

Error Code [errorCode]

- **0**: NoError
- **1**: ConnectorLockFailure
- **2**: EVCommunicationError
- **3**: GroundFailure
- **4**: HighTemperature
- **5**: InternalError
- **6**: LocalListConflict
- **7**: OtherError
- **8**: OverCurrentFailure
- **9**: OverVoltage
- **10**: PowerMeterFailure
- **11**: PowerSwitchFailure
- **12**: ReaderFailure
- **13**: ResetFailure
- **14**: UnderVoltage
- **15**: WeakSignal

Modbus Keys

- `powerImportW`: Calculate the power import and return it in W
- `powerImportK`: Calculate the power import and return it in kW
- `importVa`: Calculate the voltage ampere import and return it in VA
- `importKvA`: Calculate the voltage ampere import and return it in kVA